

**İzmir Katip Çelebi University / Computer
Engineering
CENG-302 Operating Systems (2024-2025)
Project 2**

CENG 302 – Operating Systems

Project 2 – Simulation of Classic Synchronization
Problems

Buse Berfin Halefoğlu - 210401020

Doğa Oytaç - 210401009

Nazlı Tokgöz - 210401040

Sinem Demir -210401047

04.06.2025

PART 1 – SLEEPING BARBER

1. Introduction

In modern operating systems, the efficient handling of concurrent processes is a critical concern. One of the classic problems used to model and understand process synchronization and resource sharing is the **Sleeping Barber Problem**. This problem illustrates the challenges of coordinating access to shared resources—in this case, barbers and waiting chairs—between multiple concurrent entities such as barbers and arriving customers.

In this project, we simulate the Sleeping Barber Problem to explore key synchronization mechanisms in C, utilizing **POSIX threads (pthreads)**, **semaphores**, and **mutexes**. The scenario assumes a barber shop with two barbers and five waiting chairs. A customer enters the shop, and if there is a free seat, they sit and wait to be served. If all seats are occupied, the customer leaves. The barber sleeps when no customers are present and wakes up when a customer arrives and signals readiness.

This simulation aims to ensure correct synchronization without race conditions or deadlocks while reflecting realistic behavior such as limited waiting space and concurrent barbering. The solution effectively demonstrates inter-thread communication and mutual exclusion, which are foundational concepts in operating system design.

2. Purpose of the Assignment

The main purpose of this assignment is to help us understand how **synchronization** works in operating systems by simulating a real-life inspired problem. Through the Sleeping Barber scenario, we get to practice using synchronization tools like **semaphores** and **mutexes** to manage the interaction between multiple threads.

In particular, this problem teaches us how to handle limited shared resources—in this case, waiting chairs and barbers—without causing issues like **data inconsistency**, **race conditions**, or **deadlocks**. By building this simulation ourselves, we learn not just how to write multi-threaded programs, but also how to control them so they behave correctly in all situations.

This assignment provides a hands-on opportunity to apply theoretical knowledge about process synchronization and concurrent execution, which are core concepts in operating systems. It also strengthens our ability to think logically about resource management and real-time coordination between processes.

3. Scope of the Project

In this part of the project, we focus on implementing the **Sleeping Barber Problem** using C programming with **POSIX threads**, **semaphores**, and **mutexes**. The simulation is designed around a small barbershop that has:

- **2 barbers**, each capable of serving one customer at a time

- **5 waiting chairs** for incoming customers
- **10 customers** arriving one by one

Our simulation mimics how customers interact with the shop: if there's an available seat, they wait for a barber; otherwise, they leave. The barbers stay asleep until a customer signals that they're ready. Once a haircut is done, the barber checks if someone else is waiting, and goes back to sleep if not.

This scope includes:

- Creating and managing **multiple threads** for barbers and customers
- Using **semaphores** to track waiting chairs and barber availability
- Applying **mutexes** to avoid conflicts when accessing shared data
- Ensuring proper synchronization without deadlocks or starvation
- Reflecting realistic timing with sleep intervals between customer arrivals and haircuts

The project also sets the foundation for better understanding how operating systems manage parallel tasks, such as process coordination and safe access to shared resources.

4. Code explanations

This C program simulates the classic "**Sleeping Barber Problem**", a well-known example of process synchronization in operating systems. It models a barbershop with a limited number of waiting chairs ($\text{CHAIRS} = 5$), multiple barbers ($\text{BARBERS} = 2$), and a fixed number of customers ($\text{CUSTOMERS} = 10$). It uses **POSIX threads**, **semaphores**, and **mutexes** to handle synchronization. Each customer thread tries to sit on a waiting chair; if a chair is available (`sem_trywait` succeeds), the customer waits for a barber by signaling the `barberReady` semaphore. If no chair is available, the customer leaves. Each barber thread runs indefinitely, waiting for a customer (`sem_wait` on `barberReady`), then locks the mutex to safely update the number of available chairs and simulates haircutting with a 2-second sleep. The `waitingChairs` semaphore ensures correct tracking of free chairs, while the mutex protects shared resources during updates. The main function initializes threads and semaphores, starts barber and customer threads, and joins customer threads to wait for their completion.

5. Screenshots from Execution

```

buse@DESKTOP-EUIDS5V:/mnt/c/Users/ASUS/Desktop/CENG302_Halefoglu_Buse_2$ gcc -pthread -o sleeping_barber sleeping_barber.c
./sleeping_barber
Müşteri 0 salona geldi.
Müşteri 0 bekleme koltuğuna oturdu.
Berber 0 bir müşteriyi tıraş ediyor.
Müşteri 1 salona geldi.
Müşteri 1 bekleme koltuğuna oturdu.
Berber 1 bir müşteriyi tıraş ediyor.
Müşteri 2 salona geldi.
Müşteri 2 bekleme koltuğuna oturdu.
Berber 0 bir müşteriyi tıraş ediyor.
Müşteri 3 salona geldi.
Müşteri 3 bekleme koltuğuna oturdu.
Berber 1 bir müşteriyi tıraş ediyor.
Müşteri 4 salona geldi.
Müşteri 4 bekleme koltuğuna oturdu.
Berber 0 bir müşteriyi tıraş ediyor.
Müşteri 5 salona geldi.
Müşteri 5 bekleme koltuğuna oturdu.
Berber 1 bir müşteriyi tıraş ediyor.
Müşteri 6 salona geldi.
Müşteri 6 bekleme koltuğuna oturdu.
Berber 0 bir müşteriyi tıraş ediyor.
Müşteri 7 salona geldi.
Müşteri 7 bekleme koltuğuna oturdu.
Berber 1 bir müşteriyi tıraş ediyor.
Müşteri 8 salona geldi.
Müşteri 8 bekleme koltuğuna oturdu.
Berber 0 bir müşteriyi tıraş ediyor.
Müşteri 9 salona geldi.
Müşteri 9 bekleme koltuğuna oturdu.
Berber 1 bir müşteriyi tıraş ediyor.

```

```

buse@DESKTOP-EUIDS5V:/mnt/c/Users/ASUS/Desktop/CENG302_Halefoglu_Buse_2$ ./test_script.sh
Test Script Başlatılıyor...
Kodlar Derleniyor...
Dining Philosophers Programı başarıyla derlendi.
Sleeping Barber Programı başarıyla derlendi.
Programlar çalıştırılıyor...
Dining Philosophers: Çözüm Yöntemi 1
Yöntem 1 başarıyla çalıştı ve çıktısı kaydedildi.
Dining Philosophers: Çözüm Yöntemi 2
Yöntem 2 başarıyla çalıştı ve çıktısı kaydedildi.
Dining Philosophers: Çözüm Yöntemi 3
Yöntem 3 başarıyla çalıştı ve çıktısı kaydedildi.
Dining Philosophers: Çözüm Yöntemi 4
Yöntem 4 başarıyla çalıştı ve çıktısı kaydedildi.
Sleeping Barber Programı çalıştırılıyor...
Sleeping Barber Programı başarıyla çalıştı ve çıktısı kaydedildi.
Çıktılar kontrol ediliyor...
output_philosophers_1.txt başarıyla oluşturuldu.
output_philosophers_2.txt başarıyla oluşturuldu.
output_philosophers_3.txt başarıyla oluşturuldu.
output_philosophers_4.txt başarıyla oluşturuldu.
output_barber.txt başarıyla oluşturuldu.
Test Script Tamamlandı.
buse@DESKTOP-EUIDS5V:/mnt/c/Users/ASUS/Desktop/CENG302_Halefoglu_Buse_2$ cat output_philosophers_1.txt

```

6. Source Code

```

#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

#define CHAIRS 5          // Bekleme koltuğu sayısı
#define CUSTOMERS 10     // Salona gelecek toplam müşteri sayısı
#define BARBERS 2        // Aynı anda çalışabilen berber sayısı

sem_t waitingChairs;     // boş koltuk sayısı
sem_t barberReady;       // berber hazır mı
pthread_mutex_t mutex;    // bekleme koltuğu değişimini kilitlemek için

void* customer(void* arg) {

```

```

int id = *(int*)arg;

printf("Müşteri %d salona geldi.\n", id);
fflush(stdout);
// Bekleme koltuğu varsa müşteri oturur ve berberi uyandırır
if (sem_trywait(&waitingChairs) == 0) {
    printf("Müşteri %d bekleme koltuğuna oturdu.\n", id);
    fflush(stdout);
    sem_post(&barberReady); // berberi uyandır
} else {
    // Bekleme koltuğu yoksa müşteri gider
    printf("Müşteri %d yer bulamadı, gitti.\n", id);
    fflush(stdout);
}

return NULL;
}

void* barber(void* arg) {
    int id = *(int*)arg; // Berber ID'si

    while (1) {
        sem_wait(&barberReady); // Müşteri gelene kadar bekler

        pthread_mutex_lock(&mutex); // Koltuk sayısını güvenli şekilde
// değiştirmek için kilitler
        sem_post(&waitingChairs); // Müşteri alınır, koltuk boşalır
        printf("Berber %d bir müşteriyi tıraş ediyor.\n", id);
        fflush(stdout);
        pthread_mutex_unlock(&mutex); // Kilidi bırakır

        sleep(2); // Tıraş süresi
    }

    return NULL;
}

int main() {
    pthread_t barberThreads[BARBERS];
    pthread_t customerThreads[CUSTOMERS];
    int ids[BARBERS > CUSTOMERS ? BARBERS : CUSTOMERS];

    sem_init(&waitingChairs, 0, CHAIRS);
    sem_init(&barberReady, 0, 0);
    pthread_mutex_init(&mutex, NULL);
    // Berber iş parçacıkları başlatılır
    for (int i = 0; i < BARBERS; i++) {

```

```

        ids[i] = i;
        pthread_create(&barberThreads[i], NULL, barber, &ids[i]);
    }
    // Müşteriler 1 saniye arayla salona gelir
    for (int i = 0; i < CUSTOMERS; i++) {
        ids[i] = i;
        pthread_create(&customerThreads[i], NULL, customer, &ids[i]);
        sleep(1); // her müşteri 1 sn arayla gelir
    }
    // Tüm müşteriler işini bitirene kadar ana iş parçacığı bekler
    for (int i = 0; i < CUSTOMERS; i++) {
        pthread_join(customerThreads[i], NULL);
    }

    return 0;
}

```

PART 2 – DINING PHILOSOPHERS

1. Introduction

Imagine five philosophers sitting at a round table. They think, they eat, and they share forks—but only five forks are available, and each philosopher needs two to eat. This setup might sound simple, but it actually forms the basis of one of the most well-known synchronization problems in computer science: the **Dining Philosophers Problem**.

In this project, we simulate this problem using the C programming language with **threads**, **semaphores**, and proper synchronization logic. The goal is to model how multiple "thinking and eating" processes can work together without interfering with each other or causing issues like deadlocks. The real challenge lies in ensuring that no philosopher is left starving while others eat indefinitely.

This problem, though theoretical, helps us build a solid understanding of how modern operating systems manage access to shared resources between concurrent processes.

2. Purpose of the Assignment

The purpose of this assignment is to give students practical experience with **process synchronization**—one of the fundamental concepts in operating systems. By simulating the Dining Philosophers Problem, we practice how to manage **shared resources** (the chopsticks) among competing processes (the philosophers) without causing deadlocks or starvation.

This problem provides a valuable opportunity to apply theoretical knowledge to real code using synchronization primitives such as **semaphores** and **mutexes**, and to observe how even simple design choices can lead to very different behaviors. Through this task, we aim to better understand how an operating system ensures fairness and efficiency among multiple running processes.

3. Scope of the Project

This part of the project involves creating a simulation of the Dining Philosophers Problem using **four different synchronization methods**. Each method reflects a different strategy to solve the issue of deadlock and process starvation:

1. **Maximum n-1 philosophers at the table:** Limits the number of philosophers who can try to eat at the same time.
2. **Checking chopstick availability:** Philosophers only eat if both adjacent chopsticks are free.
3. **Asymmetric solution:** Odd- and even-numbered philosophers pick up chopsticks in different orders.
4. **Right-chopstick preference:** A specific philosopher always picks the right chopstick first.

In our implementation:

- We use **threads** to represent each philosopher.
- We control access to chopsticks using **semaphores**.
- We introduce **delays (sleep)** to simulate thinking and eating times realistically.
- We design logic to avoid **deadlocks** and **race conditions** in each solution.

Each method is tested using a shell script that automates compilation, runs simulations, and checks if the output is generated correctly.

4. Code explanations

4.1 Maximum n-1 Philosophers at the Table

This solution is implemented in `philosopher()` (used when `method == 1`). The key idea is to limit the number of philosophers trying to eat concurrently to $N-1$, by using the room semaphore initialized to $N - 1$. This ensures at least one philosopher is always thinking, which prevents circular wait and therefore avoids deadlock. Each philosopher must acquire a

token from room before attempting to pick up chopsticks. If all N philosophers tried to pick up chopsticks simultaneously, a deadlock could occur if each picked up one and waited on the other. By limiting access, this risk is eliminated.

4.2 Checking Chopstick Availability

Implemented in `philosopher_method2()` (used when `method == 2`), this method uses non-blocking `sem_trywait()` to check if both chopsticks are available. A philosopher only proceeds to eat if they can acquire both chopsticks immediately; otherwise, they retry after a short delay (`usleep(100)`). This strategy avoids deadlock and resource holding by not holding onto one chopstick while waiting for another, a situation that typically leads to circular wait. It is a **busy-wait** approach that favors responsiveness over CPU efficiency.

4.3 Asymmetric Solution

This strategy is implemented in `philosopher_method3()` (used when `method == 3`). It alternates the order in which chopsticks are picked up depending on whether the philosopher has an even or odd ID. Even-numbered philosophers pick up the right chopstick first, while odd-numbered philosophers pick up the left. This breaks symmetry in the system, which is one of the main causes of deadlock. By ensuring not all philosophers compete for the same chopstick first, it prevents a circular wait condition and makes deadlock impossible.

4.4 Right Chopstick Preference

Implemented in `philosopher_method4()` (used when `method == 4`), this is a customized variation where a specific philosopher (e.g., Philosopher 2) always picks up the right chopstick first, while others follow the standard left-first approach. This intentional asymmetry introduces variation in behavior, which helps avoid deadlock in most practical cases. While not as general as the full asymmetric approach, this method demonstrates how prioritizing certain philosophers or actions can influence system behavior and prevent contention in resource acquisition.

5. Screenshots from Execution


```
buse@DESKTOP-EUIDS5V:/mnt/c/Users/ASUS/Desktop/CENG302_Halefoglu_Buse_2$ cat output_philosophers_1.txt
cat output_philosophers_2.txt
cat output_philosophers_3.txt
cat output_philosophers_4.txt
Philosopher 0 is thinking.
Philosopher 2 is thinking.
Philosopher 1 is thinking.
Philosopher 3 is thinking.
Philosopher 0 is eating.
Philosopher 2 is eating.
Philosopher 0 is thinking.
Philosopher 1 is thinking.
Philosopher 4 is thinking.
Philosopher 3 is thinking.
Philosopher 2 is thinking.
Philosopher 0 is eating.
Philosopher 3 is eating.
Philosopher 0 is thinking.
Philosopher 2 is thinking.
Philosopher 4 is thinking.
Philosopher 3 is thinking.
Philosopher 1 is thinking.
Philosopher 0 is eating.
Philosopher 2 is eating.
Philosopher 0 is thinking.
Philosopher 3 is thinking.
Philosopher 4 is thinking.
Philosopher 1 is thinking.
Philosopher 2 is thinking.
Philosopher 0 is eating.
Philosopher 3 is eating.
buse@DESKTOP-EUIDS5V:/mnt/c/Users/ASUS/Desktop/CENG302_Halefoglu_Buse_2$
```

6. Source Code

```
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>
#include <stdbool.h>

#define N 5 // Filozof sayısı ve aynı zamanda çubuk (chopstick) sayısı

sem_t room; // Yöntem 1 için en fazla N-1 filozof aynı anda masada olabilir
sem_t chopsticks[N]; // Her filozofun solunda bir çubuk (semafor)

void* philosopher(void* num) {
    int id = *(int*)num;

    sem_wait(&room); // Maksimum N-1 filozofun masaya oturmasını sağlar

    printf("Philosopher %d is thinking.\n", id);
    fflush(stdout);
    sleep(1);
    // Sol ve sağ çubuğu alır
    sem_wait(&chopsticks[id]);
    sem_wait(&chopsticks[(id + 1) % N]);
```

```

printf("Philosopher %d is eating.\n", id);
fflush(stdout);
sleep(2);
    // Yemeğini bitirir, çubukları bırakır
sem_post(&chopsticks[id]);
sem_post(&chopsticks[(id + 1) % N]);

printf("Philosopher %d finished eating and left.\n", id);
fflush(stdout);
sem_post(&room);

return NULL;
}
// Yalnızca iki çubuk da müsaitse yemeğe başlar. Aksi halde kısa bir süre
bekler ve tekrar dener.
// Bu şekilde deadlock riski en aza indirgenmiş olur.

void* philosopher_method2(void* num) {
    int id = *(int*)num;

    printf("Philosopher %d is thinking.\n", id);
    fflush(stdout);
    sleep(1);

    while (true) {
        if (sem_trywait(&chopsticks[id]) == 0) {
            if (sem_trywait(&chopsticks[(id + 1) % N]) == 0) {
                printf("Philosopher %d is eating.\n", id);
                fflush(stdout);
                sleep(2);
                printf("Philosopher %d finished eating and left.\n", id);
                fflush(stdout);

                sem_post(&chopsticks[id]);
                sem_post(&chopsticks[(id + 1) % N]);
                break;
            } else {
                sem_post(&chopsticks[id]);
            }
        }
        usleep(100);
    }

    return NULL;
}

```

```

}

void* philosopher_method3(void* num) {
    int id = *(int*)num;

    printf("Philosopher %d is thinking.\n", id);
    fflush(stdout);
    sleep(1);

    if (id % 2 == 0) {
        sem_wait(&chopsticks[(id + 1) % N]);
        sem_wait(&chopsticks[id]);
    } else {
        sem_wait(&chopsticks[id]);
        sem_wait(&chopsticks[(id + 1) % N]);
    }

    printf("Philosopher %d is eating.\n", id);
    fflush(stdout);
    sleep(2);

    sem_post(&chopsticks[id]);
    sem_post(&chopsticks[(id + 1) % N]);

    printf("Philosopher %d finished eating and left.\n", id);
    fflush(stdout);

    return NULL;
}

void* philosopher_method4(void* num) {
    int id = *(int*)num;

    printf("Philosopher %d is thinking.\n", id);
    fflush(stdout);
    sleep(1);

    if (id == 2) { // Örnek: Philosopher 2 daima sağ çubuğu önce alır
        sem_wait(&chopsticks[(id + 1) % N]);
        sem_wait(&chopsticks[id]);
    } else {
        sem_wait(&chopsticks[id]);
        sem_wait(&chopsticks[(id + 1) % N]);
    }
}

```

```

    printf("Philosopher %d is eating.\n", id);
    fflush(stdout);
    sleep(2);

    sem_post(&chopsticks[id]);
    sem_post(&chopsticks[(id + 1) % N]);

    printf("Philosopher %d finished eating and left.\n", id);
    fflush(stdout);

    return NULL;
}

int main() {
    int method;
    scanf("%d", &method);

    pthread_t threads[N];
    int ids[N];

    for (int i = 0; i < N; i++) {
        sem_init(&chopsticks[i], 0, 1);
        ids[i] = i;
    }

    if (method == 1) {
        sem_init(&room, 0, N - 1); // En fazla N-1 filozof aynı anda yemek
        // yiyebilir (deadlock önleme)
        for (int i = 0; i < N; i++)
            pthread_create(&threads[i], NULL, philosopher, &ids[i]);
    }
    else if (method == 2) {
        // Filozoflar sadece iki çubuğu birden bulabildiklerinde yemek yer
        // (sem_trywait)
        for (int i = 0; i < N; i++)
            pthread_create(&threads[i], NULL, philosopher_method2, &ids[i]);
    }
    else if (method == 3) {
        // Asimetrik çözüm: tek/çift filozoflar farklı çubuk alma sırası izler
        for (int i = 0; i < N; i++)
            pthread_create(&threads[i], NULL, philosopher_method3, &ids[i]);
    }
    else if (method == 4) {
        // Özel çözüm: yalnızca filozof 2 sağ çubuğu önce alır, diğerleri
        // standart şekilde davranır
        for (int i = 0; i < N; i++)

```

```
        pthread_create(&threads[i], NULL, philosopher_method4, &ids[i]);
    }
    else {
        // Geçersiz giriş kontrolü
        printf("Only method 1 to 4 are implemented.\n");
        fflush(stdout);
        return 0;
    }

    for (int i = 0; i < N; i++)
        pthread_join(threads[i], NULL);

    return 0;
}
```